

LoreKeeper - Resume Technique

Migration Stack RAG - Ce qui a change et pourquoi

Projet Oracle | RedDragon Games | Aethelgard Online

1. Contexte : ce que Marcus attend

Suite a la demo du projet LoreKeeper, Marcus (CTO) a valide notre approche et nous a communique la stack technique officielle de RedDragon Games. L'objectif : transformer 10 ans de donnees narratives en un Oracle intelligent avant la sortie de l'extension **Shadows of the Void**.

Ses exigences non negociables :

- Pas de notebooks en production, du code Python structure
- Type Hints clairs et tests unitaires qui valident le travail
- Pipeline robuste : avaler le chaos organique sans planter
- Anti-hallucination : ne jamais inventer d'information
- Confidentialite totale sur le projet et la stack

La stack technique imposee par Marcus :

Composant	Technologie	Role
Orchestration	LangChain + LlamaIndex	Chainer LLM, prompts, bases de donnees
Interface	Vercel AI SDK	Streaming et integration IA
Bases vectorielles	pgvector, Pinecone, Qdrant, Weaviate	Stockage et recherche de vecteurs
Embeddings	OpenAI text-embedding-3-small	Convertir texte en vecteurs mathematiques
Nettoyage	Unstructured.io, LlamaParse	Extraire du texte de fichiers complexes

2. Avant / Apres : ce qui a change

Voici la comparaison entre la version actuelle sur GitHub (branche **main**) que vous connaissez, et la nouvelle version (branche **Emir**) avec les migrations effectuees.

Composant	Avant (main / GitHub)	Apres (branche Emir)
Orchestration	Code maison (run.py)	LangChain (core, openai, text-splitters, community, qdrant)
Base vectorielle	ChromaDB (fichiers locaux)	Qdrant via langchain-qdrant (mode embarque, pret pour Cloud)
Embeddings	FastEmbed (MiniLM) via ChromaDB	FastEmbed (MiniLM) via langchain-community
Chunking	Algorithme maison (split par paragraphe)	LangChain RecursiveCharacterTextSplitter
Parsing	Parser custom uniquement (parser.py)	Unstructured.io (partition) avec fallback parser custom
Chargement fichiers	open() + lecture manuelle	LangChain Document Loaders + Unstructured.io
LLM	OpenAI client direct (hardcode DeepSeek)	LangChain ChatOpenAI (configurable via .env)
Tests	~48 tests	62 tests

Ce qui n'a PAS change :

- Le frontend (grimoire gothique en HTML/CSS/JS vanille)
- L'API Flask (routes /api/ask et /api/reindex)
- Le parser custom (parser.py) - reste en fallback
- La logique d'indexation incrementale (detection new/modified/deleted)
- Les Type Hints et la structure du code

3. Les technologies expliquees simplement

3.1 LangChain - Le framework qui connecte tout

En une phrase : LangChain est un framework Python qui sert de colle universelle entre tous les composants d'une app IA (LLM, bases de donnees, fichiers).

Analogie : C'est un adaptateur universel de prise electrique. Votre code parle LangChain, et LangChain traduit vers n'importe quel service. Si demain on change de LLM ou de base de donnees, on change une ligne, pas tout le code.

On l'utilise a 4 endroits dans le projet :

Ou ?	Module	Ce que LangChain fait
Chargement fichiers	document_loader.py	TextLoader, CSVLoader, JSONLoader remplacent nos open() manuels
Decoupage texte	chunker.py	RecursiveCharacterTextSplitter coupe intelligemment par paragraphe, ligne, phrase, mot
Base vectorielle	vector_store.py	QdrantVectorStore abstrait la connexion a Qdrant
Appel LLM	generator.py	ChatOpenAI fonctionne avec DeepSeek, OpenAI, Groq sans changer le code

Avantage principal : Si Marcus dit demain 'on passe de Qdrant a Pinecone', on remplace QdrantVectorStore par PineconeVectorStore. Le reste du code ne change pas.

3.2 Qdrant - La base de donnees vectorielle

En une phrase : Une base de donnees qui cherche par similarite de sens au lieu de chercher par mots exacts.

Analogie : En SQL classique, tu cherches un livre par son titre exact. Avec Qdrant, tu dis "je cherche quelque chose qui parle de royaute et trahison" et il trouve les 3 passages les plus proches de cette idee.

Comment ca marche :

Etape	Ce qui se passe	Exemple
1. Stockage	Le texte est transforme en vecteur (liste de 384 nombres) par FastEmbed	"Le roi Alaric gouverne" = [0.12, -0.45, 0.78, ...]
2. Question	La question est aussi transformee en vecteur par FastEmbed	"Qui dirige le royaume ?" = [0.11, -0.43, 0.80, ...]
3. Recherche	Qdrant compare les vecteurs (cosine similarity) et retourne les 3 plus proches	Top 3 passages les plus pertinents + sources

Pourquoi Qdrant et pas ChromaDB (ancien) ? Marcus l'a demande dans sa stack. Qdrant est aussi plus performant sur de gros volumes, mieux filtre par metadata, et plus utilise en production dans l'industrie.

3.3 FastEmbed - Le traducteur texte vers vecteurs

En une phrase : FastEmbed transforme du texte humain en vecteurs mathématiques (listes de nombres). Deux phrases au sens similaire auront des vecteurs proches.

Modele utilise : sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 - un modele open source de HuggingFace, leger (~150 Mo), optimise pour le francais et 100+ langues. Pas besoin de GPU.

	FastEmbed (notre choix)	OpenAI Embeddings (Marcus)
Prix	Gratuit, tourne en local	Payant (API call par texte)
Taille	~150 Mo	Cloud (rien a telecharger)
Francais	Excellent (multilingual)	Excellent
Dimensions	384	1536
Offline	Oui	Non (besoin d'internet)

3.4 Unstructured.io - Le lecteur universel de fichiers

En une phrase : Un outil qui sait lire n'importe quel type de fichier (PDF, Word, JSON, CSV, HTML, Excel...) et en extraire du texte propre.

Analogie : C'est un traducteur universel de documents. Tu lui donnes n'importe quel fichier, il te rend du texte propre. Une seule fonction `partition()`, tous les formats.

Avant vs Apres dans notre code :

	Avant (parser custom)	Apres (Unstructured.io)
Code	Une fonction par format (if .md, if .csv, if .json...)	<code>partition(fichier)</code> detecte le format tout seul
Nouveaux formats	3-4 lignes a ajouter + un import par format	0 lignes (<code>partition</code> gere tout)
Formats supportes	.md .txt .json .csv .xlsx .xml	Tous les precedents + .pdf .docx .pptx .html .eml ...
Fallback	-	Si Unstructured echoue, <code>parser.py</code> prend le relais

4. Couverture de la stack de Marcus

Ou en sommes-nous par rapport aux exigences du CTO ?

Exigence Marcus	Statut	Detail
Pas de notebooks	OK	Code Python structure en modules
Type Hints	OK	Presentes dans tout le code
Tests unitaires	OK	62 tests avec pytest + mocking
LangChain (orchestration)	OK	langchain-core, openai, text-splitters, community, qdrant
Qdrant (base vectorielle)	OK	qdrant-client en mode embarque, pret pour migration Cloud
Unstructured.io (parsing)	OK	<code>partition()</code> avec fallback custom
Anti-hallucination	OK	Prompt RAG strict, temperature 0.2
Resilience fichiers corrompus	OK	Try-catch + fallback partout
LlamaIndex (ingestion)	MANQUANT	Non integre
Vercel AI SDK (interface)	MANQUANT	On utilise Flask + vanilla JS
OpenAI Embeddings	DIFFERENT	On utilise FastEmbed (gratuit) au lieu de text-embedding-3-small
LlamaParse	MANQUANT	Non integre
pgvector / Pinecone / Weaviate	PARTIEL	Seul Qdrant est implemente

Couverture estimee : ~75% de la stack de Marcus. Les piliers (LangChain + Qdrant + Unstructured + tests + code structure) sont la. Il manque principalement LlamaIndex, les OpenAI Embeddings, et LlamaParse.

5. Prochaines etapes

Voici le plan pour la suite du projet, par ordre de priorite :

Priorite	Tache	Pourquoi	Effort
1 - CRITIQUE	Migration Qdrant Cloud	Le disque local est efface a chaque redeploy. Sans ca, on perd les embeddings.	2 lignes de code + config .env
2 - HAUTE	Deploiement Railway	Mettre en production avec Procfile + variables d'env	1-2 heures
3 - HAUTE	Interface Admin	Upload fichiers, reindex, liste des fichiers indexes (page /admin protegee)	4-6 heures
4 - MOYENNE	Fixer python-magic sur Windows	Unstructured crash sur Windows (segfault python-magic). Fonctionne sur Linux (prod).	pip install python-magic-bin
5 - OPTIONNEL	Integrer OpenAI Embeddings	Passer de FastEmbed a text-embedding-3-small comme Marcus le demande	Modifier vector_store.py + cle API OpenAI

6. Architecture actuelle du pipeline

Le schema complet est disponible dans le fichier `architecture-lorekeeper.drawio` (ouvrir sur app.diagrams.net). Voici le flux simplifie :

Phase 1 : Ingestion des donnees

Etape	Module	Technologie
1. Lecture fichiers	<code>document_loader.py</code>	Unstructured.io <code>partition()</code>
2. Nettoyage	<code>parser.py</code>	<code>clean_text()</code> (regex, variables jeu)
3. Decoupage	<code>chunker.py</code>	LangChain <code>RecursiveCharacterTextSplitter</code>
4. Vectorisation	<code>vector_store.py</code>	FastEmbed (MiniLM multilingual)
5. Stockage	<code>vector_store.py</code>	Qdrant (collection 'lore')

Phase 2 : Requete utilisateur (RAG)

Etape	Module	Ce qui se passe
1. Question	<code>routes.py</code>	POST <code>/api/ask</code> recoit la question
2. Recherche	<code>search.py</code>	Vectorise la question, cherche les 3 passages les plus proches
3. Generation	<code>generator.py</code>	LangChain <code>ChatOpenAI</code> genere une reponse basee sur le contexte

Etape	Module	Ce qui se passe
4. Reponse	routes.py	JSON {reponse, sources} renvoye au frontend